

Relacion-Temas-1-y-2-Sistemas-Op...



ricardoortzz



Sistemas Operativos



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

70 años formando talento
que transforma el futuro.

La primera escuela de negocios de España,
hoy líder en sostenibilidad y digitalización.



EOI Escuela de
organización
industrial



Descubre EOI



Ver sin gafas ni lentillas es otro rollo

Empieza el día sin lóos: olvídate de gafas y lentillas para siempre.



Consulta + cirugía top en Granada. ¡Haz match con tu mejor visión!

Relación Temas 1 y 2 (Sistemas Operativos).

1. Cuestiones sobre procesos, y asignación de CPU:

a) ¿Es necesario que lo último que haga todo proceso antes de finalizar sea una llamada al sistema para finalizar? ¿Sigue siendo esto cierto en sistemas monoprogramados?

Sí, normalmente lo último que hace un proceso antes de finalizar es una llamada al sistema (por ejemplo, *exit*) para que el sistema operativo libere sus recursos.

En sistemas monoprogramados sigue siendo necesario, ya que el SO es el único que puede gestionar y liberar los recursos de forma segura.

b) Cuando un proceso se bloquea, ¿deberá encargarse él directamente de cambiar el valor de su estado en el descriptor de proceso o PCB?

No. El proceso no modifica directamente su propio estado en el PCB. El cambio de estado lo realiza el sistema operativo (modo kernel) tras realizar una llamada al sistema.

c) ¿Qué debería hacer el planificador a corto plazo cuando es invocado pero no hay ningún proceso en la cola de ejecutables?

Debe poner a la CPU en estado inactivo (modo *idle*) o ejecutar un proceso nulo hasta que haya algún proceso listo. Esto evita el desperdicio de recursos y permite al sistema reactivarse cuando llegue un evento o interrupción.

d) ¿Qué algoritmos de planificación quedan descartados para ser utilizados en sistemas de tiempo compartido?

Quedan descartados los algoritmos no apropiativos (como *FCFS* o *SJF no apropiativo*), porque no permiten que el sistema quite la CPU a un proceso en ejecución. En tiempo compartido se requieren algoritmos apropiativos, como *Round Robin* o *SRTF*, para asegurar una respuesta rápida e interacción justa entre usuarios.

2. La representación gráfica del cociente $[(\text{tiempo_en_cola_ejecutables} + \text{tiempo_de_CPU}) / \text{tiempo_de_CPU}]$ frente a tiempo_de_CPU suele mostrar valores muy altos para ráfagas muy cortas en casi todos los algoritmos de asignación de CPU. ¿Por qué?

Porque al tener ráfagas cortas (tiempo de CPU), la proporción entre el tiempo de finalización (t^o en cola de ejecutables + t^o de CPU) y el tiempo de CPU se hace muy grande. Esto provoca que matemáticamente tengamos un dividendo mayor que el divisor, dando así resultados altos. Por tanto, los procesos con ráfagas cortas, sufren más el impacto relativo de la espera, pues pasan más tiempo en la cola que en la CPU.



¡Escanea!

WUOLAH

3. Para cada una de las llamadas al sistema siguientes, especificar y explicar si su procesamiento por el sistema operativo requiere la invocación del planificador a corto plazo:

a) Crear un proceso.

No necesariamente. El SO crea el PCB y lo pone en la cola de listos. Si el planificador fuera apropiativo (por ej. por prioridad) y el nuevo proceso tuviera mayor prioridad que el actual, si sería necesaria la invocación del planificador.

b) Abortar un proceso, es decir, terminarlo forzosamente.

Si. Dado que la CPU es liberada, es necesario invocar al planificador para que seleccione un nuevo proceso de la cola de listos para ocupar la CPU.

c) Suspender o bloquear un proceso.

Si, por la misma razón que antes.

d) Reanudar un proceso (inversa al caso anterior).

No necesariamente. Solo sería necesario si el planificador es apropiativo y el proceso que se va a reanudar tiene una prioridad más alta.

e) Modificar la prioridad de un proceso.

Si el sistema es apropiativo, si, pues si dicha prioridad es modificada a una más alta, el planificador debe sacar de la CPU al proceso actual y asignarle el nuevo con prioridad más alta.

4. Sea un sistema multiprogramado que utiliza el algoritmo Por Turnos (Round-Robin). Sea S el tiempo que tarda el despachador en cada cambio de contexto. ¿Cuál debe ser el valor de quantum Q para que el porcentaje de uso de la CPU por los procesos de usuario sea del 80%?

Llamamos *Tiempo total por ciclo* a la suma de $S + Q$.

Por tanto, la utilización de la CPU se define como: $Utilización = \frac{Tiempo\ útil\ (Q)}{Tiempo\ total\ por\ ciclo}$.

Dado que se nos dice que el porcentaje de uso de CPU debe ser del 80% (0.8) $\Rightarrow$

$$0.8 = \frac{Q}{Q+S} \Rightarrow 0.8(Q + S) = Q \Rightarrow 0.8Q + 0.8S = Q \Rightarrow 0.8S = 0.2Q \Rightarrow Q = 4S$$

Es decir, el valor del quantum debe ser 4 veces el tiempo que tarda el despachador en hacer un cambio de contexto.

5. Sea un sistema multiprogramado que utiliza el algoritmo Por Turnos (Round-Robin). Sea S el tiempo que tarda el despachador en cada cambio de contexto, y N el número de procesos existente. ¿Cuál debe ser el valor de quantum Q para que se asegure que cada proceso “ve” la CPU al menos cada T segundos?

Definimos como $N \times Q$ el tiempo de ejecución de N procesos y como $N \times S$ el tiempo de cambio de contexto (*Overhead*) de N procesos.

Por tanto, el tiempo de ciclo de retorno = $N \times Q + N \times S$

Para que cada proceso “vea” la CPU al menos cada T segundos, el tiempo de ciclo de retorno debe ser menor o igual a T . Para calcular Q , usamos la siguiente igualdad: $N \times Q + N \times S = T$

Por ello:

$$N \times Q = T - N \times S \Rightarrow Q = \frac{T - N \times S}{N} \Rightarrow Q = \frac{T}{N} - \frac{N \times S}{N} \Rightarrow Q = \frac{T}{N} - S$$

Y este último representa el valor máximo del quantum, permitido. A su vez, y dado que Q debe ser positivo, se requiere que el cociente entre la frecuencia de “visualización” (T) y el número de procesos sea mayor que el tiempo del despachador.

6. ¿Tiene sentido mantener ordenada por prioridades la cola de procesos bloqueados? Si lo tuviera, ¿en qué casos sería útil hacerlo?

No tiene mucho sentido, pues un proceso bloqueado no compite por la CPU, sino que está esperando a un evento específico.

Sería útil en colas condicionales (Mutex o semáforos), porque cuando recurso exclusivo se libera, varios procesos pueden estar esperando su liberación, por lo que en lugar de despertar a todos o elegir a dedo, el sistema puede despertar a aquel con mayor prioridad. También podría ser útil si dos o más procesos estuvieran esperando a que se produjera el mismo evento u operación E/S.

7. ¿Puede el procesador manejar una interrupción mientras está ejecutando un proceso si la política de planificación que utilizamos es no apropiativa?

Si, el manejo de interrupciones es obligatorio y es independiente del tipo de planificación, pues una interrupción es una señal asíncrona (usualmente hardware) que indica que la CPU debe detener temporalmente su ejecución actual para manejar un evento (siempre en modo kernel). Esto es independiente de la política de planificación, pues esta se refiere a la regla que sigue el planificado a corto plazo al seleccionar qué proceso de usuario se ejecutará a continuación.



Ver sin gafas ni lentillas es otro rollo

Empieza el día sin lóos: olvídate de gafas y lentillas para siempre.



Consulta + cirugía top en Granada. ¡Haz match con tu mejor visión!

8. Suponga que es responsable de diseñar e implementar un sistema operativo que va a utilizar una política de planificación apropiativa. Suponiendo que tenemos desarrollado el algoritmo de planificación a tal efecto, ¿qué otras partes del sistema operativo habría que modificar para implementar tal sistema? y ¿cuáles serían tales modificaciones?

Al implementar una política apropiativa, se requiere modificar funcionalidades en ciertas partes del kernel relacionadas con el control de procesos y la gestión de interrupciones. Por ejemplo:

- Reloj hardware y rutina de servicio de interrupción del reloj. Debe configurarse para generar una interrupción periódica (quantum). Al dispararse la interrupción debe marcar al proceso actual como listo y lo devuelva a la cola de listos. También debe invocar al planificador para seleccionar al siguiente proceso
- Módulo del despachador. Se debe implementar la funcionalidad de cambio de contexto en ambos sentidos y que sea capaz de hacerlo en respuesta a interrupciones. Cuando el planificador es invocado, el despachador debe guardar el estado completo en su PCB y cargar el estado del nuevo proceso seleccionado.
- Estructura del PCB y mecanismos de sincronización. Se debe asegurar que el PCB almacene de forma segura todos los registros de la CPU, PC y estado de la pila. También se deben asegurar las regiones críticas del kernel (PCB o colas) y no puedan ser modificadas simultáneamente por la rutina de interrupción y el código principal del kernel. Requiere el uso de bloqueos de interrupciones (*spinlocks*).
- Llamadas al sistema bloqueantes y de creación/prioridad. Las llamadas que fuerzan al proceso a bloquearse deben invocar al despachador inmediatamente después de cambiar el estado del proceso, mientras que las que crean un nuevo proceso o modifican prioridad, deben invocar al planificador para que evalúe si el nuevo proceso debe apropiarse de la CPU inmediatamente.

9. En el algoritmo de planificación FCFS, la penalización $((t + t^o \text{ de espera}) / t)$, ¿es creciente, decreciente o constante respecto a t (tiempo de servicio de CPU requerido por un proceso)? Justifique su respuesta.

Si t es muy pequeño (ráfagas cortas), el denominador es cercano a cero, haciendo que el cociente sea muy grande y resulte en una alta penalización. En cambio, si es muy grande, hace que el cociente sea pequeño, resultando en una baja penalización (más cercana a 1).

Por tanto, y como hemos dicho, a medida que t aumenta, disminuye el resultado, por lo que la penalización es decreciente respecto a t .

En cuanto al algoritmo *FCFS*, este hace que los procesos largos que llegan primero penalicen a los procesos cortos que lleguen después, ya que estos tienen un tiempo de espera grande, disparando el cociente de penalización.



¡Escanea!

WUOLAH

10. En la tabla siguiente se describen cinco procesos:

Proceso	Tiempo de creación	Tiempo de CPU
A	4	1
B	0	5
C	1	4
D	8	3
E	12	2

Si suponemos que tenemos un algoritmo de planificación que utiliza una política FIFO (primero en llegar, primero en ser servido), calcula:

a) Tiempo medio de respuesta.

El tiempo de respuesta se define como el tiempo que tarda un proceso en ser atendido por la CPU por primera vez. Teniendo en cuenta que tenemos un algoritmo FCFS:

Proceso A: $T = 5$. Llega en el tiempo 4 y empieza a ejecutarse en el 9, que es cuando C finaliza.

Proceso B: $T = 0$. Por ser el primero en llegar

Proceso C: $T = 4$. Llega en el tiempo 1 y empieza a ejecutarse en el 5, que es cuando B finaliza.

Proceso D: $T = 2$. Llega en el tiempo 8 y empieza a ejecutarse en el 10, que es cuando A finaliza.

Proceso E: $T = 1$. Llega en el tiempo 12 y empieza a ejecutarse en el 13, que es cuando D finaliza.

$$\text{Media: } \frac{5+0+4+2+1}{5} = 2'4.$$

b) Tiempo medio de espera.

El tiempo de espera se define como el tiempo que ha permanecido el proceso en la cola de ejecutables. Se calcula como la diferencia entre el tiempo de finalización (diferencia entre el momento en el que finaliza y el momento en el que llega) y el tiempo de CPU. Teniendo en cuenta que es un algoritmo no apropiativo, coincide con el tiempo de respuesta. Aún así, se demuestra dicha propiedad con los siguientes cálculos

$$\text{Proceso A: } M = (10 - 4) - 1 = 5.$$

$$\text{Proceso B: } M = (5 - 0) - 5 = 0.$$

$$\text{Proceso C: } M = (9 - 1) - 4 = 4.$$

$$\text{Proceso D: } M = (13 - 8) - 3 = 2.$$

$$\text{Proceso E: } M = (15 - 12) - 2 = 1.$$

$$\text{Media} = 2'4.$$

c) La penalización, es decir, el cociente entre el tiempo de respuesta y el tiempo de CPU.

$$\text{Proceso A: } P = 5 / 1 = 5.$$

$$\text{Proceso B: } P = 0 / 5 = 0.$$

$$\text{Proceso C: } P = 4 / 4 = 1.$$

$$\text{Proceso D: } P = 2 / 3 = 0'67.$$

$$\text{Proceso E: } P = 1 / 2 = 0'5.$$



Ver sin gafas ni lentillas es otro rollo

Empieza el día sin líos: olvídate de gafas y lentillas para siempre.



RODRÍGUEZ LÓPEZ
CONSULTA OFTALMOLÓGICA



Consulta + cirugía top en Granada.
¡Haz match con tu mejor visión!



¡Escanea!

11. Utilizando los valores de la tabla del problema anterior, calcula los tiempos medios de espera y respuesta para los siguientes algoritmos:

a) Por Turnos con quantum $q=1$.

Debemos tener en cuenta que ahora se nos presenta un algoritmo de planificación apropiativo, por lo que el tiempo de espera no se corresponderá con el de espera..

Proceso A : $T = 2$. Dado que la cola interna del algoritmo *Round-Robin* presenta una filosofía *FIFO*, cada vez que un proceso agota su quantum o es creado, se añade al final de la cola. Esto hace que durante los segundos 1 a 4 se vayan turnando los procesos B y C . Para el momento en el que llega A se está ejecutando el proceso B , que tiene como siguiente proceso a C y posteriormente A , que ha sido el último en llegar. Por tanto, en el momento 5 se ejecuta C y en el 6, A .

Proceso B : $T = 0$. Por ser el primero en llegar.

Proceso C : $T = 0$. Dado que el quantum es 1, el proceso B solo da una ráfaga, justo cuando entra C .

Proceso D : $T = 2$. Siguiendo el razonamiento del proceso A , en este momento tenemos 2 procesos (B y C) pues A ya ha finalizado (en el momento 7). El proceso C da su última ráfaga cuando llega el proceso D , y tras C iba en la cola el proceso B . Por ende, C finaliza en el momento 9 y B en el momento 10, ejecutándose D en el mismo momento.

Proceso E : $T = 0$. Dado que queda solo el proceso D por finalizar, este ejecuta 2 ráfagas seguidas hasta que llega E , pues suponemos que la llegada de un proceso ocurre antes que la interrupción de otro. Por tanto, E ejecuta su ráfaga en el momento 12, y finaliza en el momento 15, tras haber finalizado D en el momento 14.

$$\text{Proceso } A: \quad M = (7 - 4) - 1 = 2.$$

$$\text{Proceso } B: \quad M = (10 - 0) - 5 = 5.$$

$$\text{Proceso } C: \quad M = (9 - 1) - 4 = 4.$$

$$\text{Proceso } D: \quad M = (14 - 8) - 3 = 3.$$

$$\text{Proceso } E: \quad M = (15 - 12) - 2 = 1.$$

$$\text{Media } T: 2 / 5 = 0,4.$$

$$\text{Media } M: 15 / 5 = 3.$$



Ver sin gafas ni lentillas es otro rollo

Empieza el día sin líos: olvídate de gafas y lentillas para siempre.



Consulta + cirugía top en Granada. ¡Haz match con tu mejor visión!

b) Por Turnos con quantum $q=4$.

Proceso A: $T = 4$. El proceso C finaliza en el momento 8 (a la misma vez que agota su quantum), y dado que consideramos que la llegada de un proceso nuevo se produce antes que el agotamiento del quantum de otro proceso (en este caso B, que acabó su quantum en 4), A se encuentra por delante en la cola, empezando a ejecutarse pues en el momento 8.

Proceso B: $T = 0$. Por ser el primero en llegar.

Proceso C: $T = 3$. Dado que el quantum es 4, el proceso B solo da 4 ráfagas, y C, que ha entrado en el momento 1, empieza a ejecutarse en el 4.

Proceso D: $T = 2$. Tras ejecutar A una sola ráfaga (finaliza en el momento 9), el proceso B finaliza su ejecución en el momento 10, dando lugar a la ejecución de D en el momento 10.

Proceso E: $T = 1$. El proceso D da sus tres ráfagas de CPU antes de agotar su quantum (finalizando en el momento 13), dando paso a la ejecución del proceso E en el momento 13 y por tanto, finalizando en el 15.

$$\text{Proceso A: } M = (9 - 4) - 1 = 4.$$

$$\text{Proceso B: } M = (10 - 0) - 5 = 5.$$

$$\text{Proceso C: } M = (8 - 1) - 4 = 3.$$

$$\text{Proceso D: } M = (13 - 8) - 3 = 2.$$

$$\text{Proceso E: } M = (15 - 12) - 2 = 1.$$

$$\text{Media } T: 10 / 5 = 2.$$

$$\text{Media } M: 15 / 5 = 3.$$

c) El más corto primero (SJF). Suponga que se estima una ráfaga igual a la real.

Debemos tener en cuenta que se nos presenta un algoritmo de planificación no apropiativo. Por tanto, los tiempos de espera y respuesta coincidirán y sus medias también. En este caso, se ejecutará el proceso con menos tiempo de CPU primero.

Proceso A: $T = 1$. En el momento en el que B finaliza (momento 5) en la cola están tanto A como C, siendo A el que menos ráfagas de CPU tiene.

Proceso B: $T = 0$. Por ser el primero en llegar.

Proceso C: $T = 5$. El proceso A termina en el momento 6 y dado que no hay más procesos en la cola, el proceso C comienza su ejecución en dicho momento.

Proceso D: $T = 2$. El proceso C termina en el momento 10 y dado que no hay más procesos en la cola, el proceso D comienza su ejecución en dicho momento.

Proceso E: $T = 1$. El proceso D termina en el momento 13 y dado que no hay más procesos en la cola, el proceso E comienza su ejecución en dicho momento (terminará en el momento 15).

$$\text{Proceso A: } M = (6 - 4) - 1 = 1.$$

$$\text{Proceso B: } M = (10 - 0) - 5 = 5.$$

$$\text{Proceso C: } M = (10 - 1) - 4 = 5.$$

$$\text{Proceso D: } M = (13 - 8) - 3 = 2.$$

$$\text{Proceso E: } M = (15 - 12) - 2 = 1.$$

$$\text{Media } T: 10 / 5 = 2.$$

$$\text{Media } M: 15 / 5 = 3.$$



¡Escanea!

WUOLAH

13. Calcula el tiempo de espera medio para los procesos de la tabla utilizando el algoritmo: el primero más corto apropiativo (o primero el de tiempo restante menor, SRTF).

Proceso	Tiempo de creación	Tiempo de CPU
A	0	3
B	1	1
C	3	12
D	9	5
E	12	5

Este algoritmo se basa en que, si se crea un proceso con menor tiempo de CPU que el actual, se apropiará de la CPU. Dado que es un algoritmo apropiativo, el tiempo de espera **NO** coincide con el tiempo de respuesta. Por tanto, vamos a calcular el tiempo de retorno o finalización de cada proceso:
 Proceso A: $T^{\circ} \text{finalización} = 4 - 0 = 0$. Se empieza a ejecutar el primero pero solo realiza una ráfaga, pues después llega el proceso B, el cual tiene menor tiempo de CPU. Luego vuelve a ejecutarse A pues B ya ha finalizado en el momento 2, finalizando A en el momento 4, pues aunque haya llegado C, tiene mucho más tiempo de CPU.

Proceso B: $T^{\circ} \text{finalización} = 2 - 1 = 1$. Explicado en el apartado anterior

Proceso C: $T^{\circ} \text{finalización} = 26 - 3 = 13$. Este proceso comienza a ejecutarse en el momento 4, cuando finaliza A y realiza 5 ráfagas (7 restante), pues posteriormente llega D con menor tiempo de CPU, que ejecuta todas sus ráfagas, pues al llegar E, aunque tengan el mismo tiempo de CPU, el proceso D se mantiene, finalizando en el momento 14. Después el proceso E ejecuta sus 5 ráfagas por tener menos tiempo de CPU que el proceso C, finalizando así en el momento 19. Por último, C ejecuta sus últimas 7 ráfagas de manera continua, terminando así en el momento 26.

Proceso D: $T^{\circ} \text{finalización} = 14 - 9 = 5$. Explicado en el apartado anterior.

Proceso E: $T^{\circ} \text{finalización} = 19 - 12 = 7$. Explicado en el apartado del proceso C.

Media = $26 / 5 = 5'2$.

13. Utilizando la tabla del ejercicio anterior, dibuja el diagrama de ocupación de CPU para el caso de un sistema que utiliza un algoritmo de colas múltiples con realimentación con las siguientes colas:

Cola	Prioridad	Quantum
1	1	1
2	2	2
3	3	4

y suponiendo que:

- Todos los procesos inicialmente entran en la cola de mayor prioridad (menor valor numérico). Cada cola se gestiona mediante la política Por Turnos.
- la política de planificación entre colas es por prioridades no apropiativo.
- un proceso en la cola i pasa a la cola i+1 si consume un quantum completo sin bloquearse.
- cuando un proceso llega a la cola de menor prioridad, permanece en ella hasta que finalice.

Tiempo	CPU	Cola 1	Cola 2	Cola 3	Observación
0-1	A	-	-	-	-
1-2	B	-	A	-	A agota Q (2rest)
2-3	A	-	-	-	-
3-4	A	C	-	-	A finaliza
4-5	C	-	-	-	-
5-6	C	-	-	-	C agota Q (11rest)
6-7	C	-	-	-	-
7-8	C	-	-	-	C agota Q (9rest)
8-9	C	-	-	-	-
9-10	C	D	-	-	-
10-11	C	D	-	-	-
11-12	D	-	-	C	C agota Q (5rest)
12-13	E	-	D	C	D agota Q (4rest)
13-14	D	-	E	C	E agota Q (4rest)
14-15	D	-	E	C	-
15-16	E	-	-	C - D	D agota Q (2rest)
16-17	E	-	-	C - D	-
17-18	C	-	-	D - E	E agota Q (2rest)
18-19	C	-	-	D - E	-
19-20	C	-	-	D - E	-
20-21	C	-	-	D - E	-
21-22	D	-	-	E - C	C agota Q (1rest)
22-23	D	-	-	E - C	-
23-24	E	-	-	C	D finaliza
24-25	E	-	-	C	-
25-26	C	-	-	-	E finaliza
26-27	-	-	-	-	C finaliza

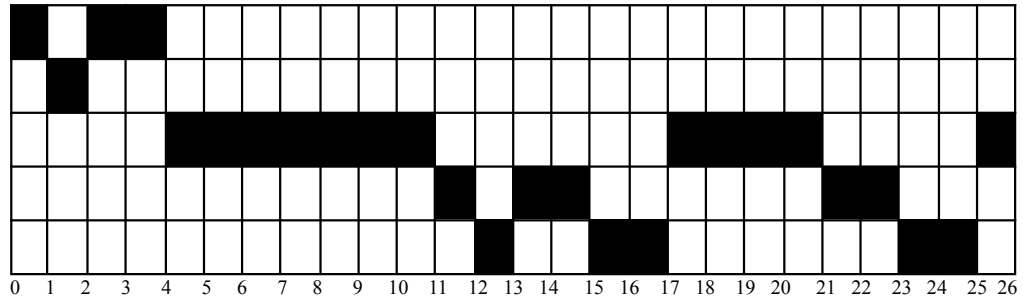


Ver sin gafas ni lentillas es otro rollo

Empieza el día sin líos: olvídate de gafas y lentillas para siempre.



Consulta + cirugía top en Granada. ¡Haz match con tu mejor visión!



Donde la primera fila corresponde al proceso *A*, la segunda al proceso *B*, la tercera al proceso *C*, la cuarta al proceso *D*, y la quinta al proceso *E*.

14. Suponga que debe maximizar la eficiencia de un sistema multiusuario y que está recibiendo quejas de muchos usuarios sobre los pobres tiempos de respuesta (o tiempos de vuelta) de sus procesos. Los resultados obtenidos con una herramienta de monitorización del sistema nos muestran que la CPU se utiliza al 99'9% de su tiempo y que los procesadores de E/S están activos solo un 10% de su tiempo. ¿Cuales pueden ser las razones de estos tiempos de respuesta pobres y por qué?

a) El quantum en la planificación Round-Robin es muy pequeño.

El cambio constante de cambios de estado puede saturar la CPU, pero no necesariamente satura la entrada y salida. Solo podría darse si el proceso es muchísimo más pequeño en proporción con el quantum.

b) La memoria principal es insuficiente.

Este es el caso más probable, pues el alto uso de CPU y tan bajo de E/S, indica que probablemente esté ocurriendo *thrashing*, que consiste en el intento desesperado de traer páginas del disco debido a la falta de RAM.

c) El sistema operativo tiene que manejar mucha memoria principal por lo que las rutinas de gestión de memoria están consumiendo todos los ciclos de CPU.

Esto sucedería si la memoria principal fuera muy grande, pues el aumento de memoria principal conlleva al aumento de la complejidad y el tiempo necesario para buscar en las estructuras de gestión de memoria, lo que hace casi imposible que se sature la CPU al haber tanta RAM.

d) La CPU es muy lenta.

El hecho de que la CPU sea lenta, influye principalmente en los tiempos de respuesta, no en la saturación de la misma. Podría darse si los procesos limitados por CPU se ejecutan de manera muy lenta.

e) El quantum en la planificación Round-Robin es muy grande.

Aumenta el tiempo de respuesta promedio, pero no necesariamente tiene por qué saturar la CPU mientras se utiliza la E/S.



¡Escanea!

WUOLAH

15. Compare el rendimiento ofrecido al planificar el conjunto de tareas multi-hebras descrito en la tabla y bajo las siguientes configuraciones:

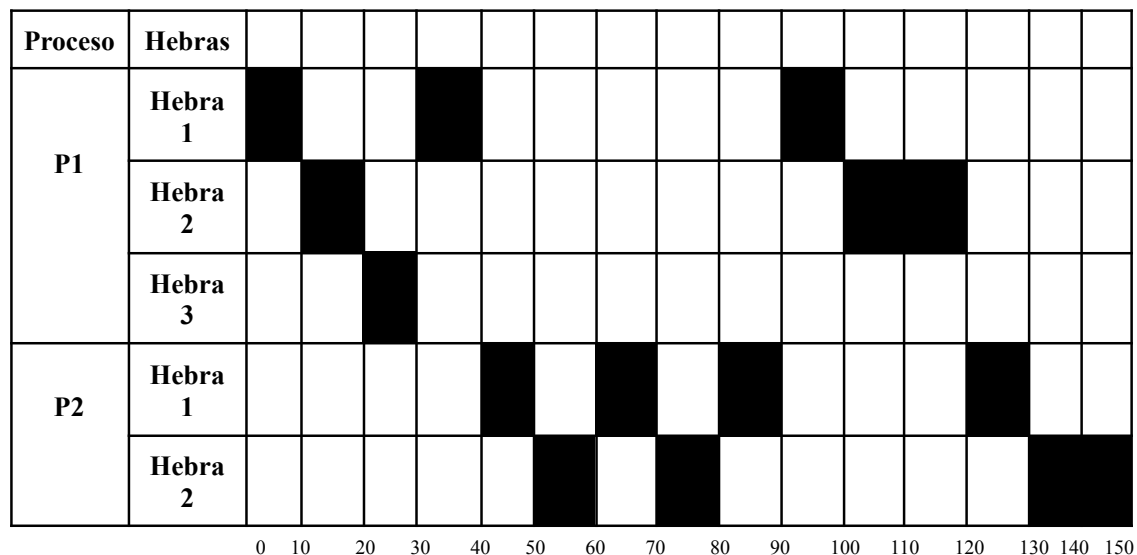
a) Sistema operativo multiprogramado con hebras de usuario. En este sistema se dispone de una biblioteca para la programación con hebras en el espacio de usuario. El algoritmo de planificación de CPU utilizado por el SO es Round-Robin con un quantum de 50 u.t. (unidades de tiempo). El planificador de la biblioteca de hebras reparte el quantum del proceso (tarea) entre las hebras utilizando Round-Robin con un quantum para cada hebra de 10 u.t. Suponga que no existe coste en el cambio de contexto entre hebras ni entre procesos.

b) Sistema operativo multiprogramado con hebras kernel. El SO planifica las hebras usando Round-Robin con un quantum de 10 u.t. Como en el apartado anterior, suponga que no existe coste en la operación de cambio de contexto. Considere además que las operaciones de E/S de un proceso únicamente bloquean a la hebra que las solicita. Suponga en ambos casos que los dos procesos están disponibles y que el planificador entrega la CPU al proceso P1. Para realizar la comparación represente en cada caso el diagrama de ocupación de CPU y calcule el grado de ocupación de la CPU (tiempo CPU ocupada \ tiempo total).

Proceso	Hebras	Comportamiento		
		Ráfaga de CPU	Tiempo de E/S	Ráfaga de CPU
P1	Hebra 1	20	30	10
	Hebra 2	30	-	-
	Hebra 3	10	-	-
P2	Hebra 1	30	30	10
	Hebra 2	40	-	-

Debemos tener en cuenta que si una hebra de usuario se bloquea por E/S, todo el proceso se bloqueará. Mientras tanto, si una hebra de kernel se bloquea por E/S, solo se bloquea dicha hebra, pero el proceso se sigue ejecutando.

Con hebras de usuario:



Con hebras kernel:

Proceso	Hebras	0	10	20	30	40	50	60	70	80	90	100	110	120	130	140	150
P1	Hebra 1	■					■				■						
	Hebra 2		■					■				■					
	Hebra 3			■													
P2	Hebra 1				■				■					■			■
	Hebra 2					■				■					■	■	

Cálculos de los grados de ocupación (índice de respuesta):

Hebras de usuario: $R = 150 / 150 = 1 \rightarrow 100\%$

Hebras kernel: $R = 150 / 150 = 1 \rightarrow 100\%$

En ambos casos, la CPU está ocupada al 100%. Lo que sí notamos es que al ejecutar con hebras kernel, la ejecución es más organizada.

16. ¿El planificador CFS de Linux favorece a los procesos limitados por E/S (cortos) frente a los procesos limitados por CPU (largos)? Explique cómo lo hace.

El planificador *Completely Fair Scheduler* favorece indirectamente a los procesos limitados por E/S manteniendo un registro del tiempo de ejecución virtual (*vruntime*) de cada proceso, con el objetivo de que todos los procesos tengan uno similar. Aquel que lo tenga menor, será siempre el siguiente en ejecutarse. De esta forma, dado que un proceso limitado por CPU utiliza continuamente su tiempo asignado, su tiempo de ejecución virtual aumenta rápidamente a medida que se ejecuta, por lo que es menos propenso a ser seleccionado por el planificador.

Por otra parte, el proceso limitado por E/S solo se ejecuta por un corto periodo de tiempo antes de bloquearse. Mientras lo está, su *vruntime* no aumenta, haciendo que sea menor y por tanto más propenso a ser seleccionado por el planificador.

17. ¿Cuál es el problema que se plantea en Linux cuando un proceso no realiza la llamada al sistema `wait` para cada uno de sus procesos hijos que han terminado su ejecución? ¿Qué efecto puede producir esto en el sistema?

Si un proceso padre no realiza la llamada al sistema `wait()` se produce la creación de procesos Zombie (ó *defunct*), que son procesos que han completado su ejecución pero su entrada en la tabla de procesos (PCB) se mantiene. Esto hace que se pueda producir una acumulación de procesos zombie en el sistema, agotando así las entradas en la tabla de procesos (PID) e impidiendo que se puedan crear nuevos procesos, requiriendo un servicio del sistema, que reasigna a los zombies al proceso *init* o *systemd* para que sean limpiados.



Ver sin gafas ni lentillas es otro rollo

Empieza el día sin lóos: olvídate de gafas y lentillas para siempre.



Consulta + cirugía top en Granada. ¡Haz match con tu mejor visión!

18. La orden clone sirve tanto para crear un proceso en Linux como una hebra.

a) Escriba los argumentos que debería tener clone para crear un proceso y una hebra.

b) Dibuje las principales estructuras de datos del kernel que reflejan las diferencias entre ambas.

La sintaxis general es la siguiente:

```
int clone(int (*fn)(void *), void *child_stack, int flags, void *arg);
```

La diferencia clave entre crear un proceso y una hebra reside en los *flags*. Estos especifican qué recursos debe compartir el nuevo proceso o hebra con su padre.

Para crear un proceso tradicional, el hijo debe tener copias privadas de los recursos del padre. El conjunto de *flags* debe especificar que no se comparte casi nada. Para una hebra el hijo debe compartir la mayoría de los recursos lógicos con su padre, lo que permite que la hebra trabaje sobre el mismo estado de memoria.

Tanto los procesos como las hebras, son representados internamente por la misma estructura de datos del kernel:

Estructura	Función	Proceso	Hebra
<i>task_struct</i>	Bloque de control. Contiene PID, estado de ejecución y punteros a otras estructuras.	Cada uno tiene la suya.	Cada una tiene la suya.
<i>mm_struct</i>	Describe el espacio de direcciones de memoria virtual.	El hijo obtiene una copia de esta estructura.	Puntero compartido a la misma <i>mm_struct</i> del padre.
<i>files_struct</i>	Describe los descriptores de archivos abiertos.	El hijo obtiene una copia de esta estructura.	Puntero a la misma <i>files_struct</i> del padre.
<i>fs_struct</i>	Describe el sistema de archivos	El hijo obtiene una copia.	Puntero compartido a la misma <i>fs_struct</i> .



¡Escanea!

WUOLAH